

学号：202300130183	姓名：宋浩宇
实验名称：实验七：词义相似度基础实验	
1. 实验目的	
1. 理解语义分析的基本概念及其与词汇分析的区别。 2. 理解词语语义相似度的含义及其计算方法。 3. 掌握词向量（Word Embedding）的基本思想。 4. 学习使用 Python 工具（gensim）计算词语相似度。 5. 能够分析不同词语之间的语义关系是否符合直觉。	
2. 实验原理	
语义分析关注语言单位的“意义”及其关系。词语之间不仅有形式差异，还存在语义距离。 例如：“高兴”和“快乐”语义接近，而“高兴”和“悲伤”语义相反。 词向量方法将词语表示为向量，使语义相似的词在向量空间中距离更近。 常用相似度计算方法： 1. 余弦相似度：衡量两个向量夹角 2. 欧氏距离：衡量空间距离 本实验通过词向量计算词语之间的相似度，并分析结果。	
3. 实验步骤与代码	
实验步骤： 1. 导入相关库（gensim、numpy 等） 2. 加载词向量模型（或训练简单模型） 3. 输入待比较的词对 4. 获取词向量表示 5. 计算余弦相似度 6. 输出相似度结果 7. 对结果进行排序和分析	
代码： <pre># -*- coding: utf-8 -*- import os from pathlib import Path import jieba import numpy as np from gensim.models import KeyedVectors, Word2Vec from sklearn.metrics.pairwise import cosine_similarity # 预训练词向量路径(Word2Vec 文本/二进制格式, 或 .kv);不存在则自动用语料训练 PRETRAINED_W2V_PATH = os.environ.get("PRETRAINED_W2V_PATH", "./pretrained-zh.w2v.txt") PAIRS_PATH = r"./exp-4.1-data.txt" CORPUS_PATH = r"./exp-4.1-corpus.txt"</pre>	

```

def load_word_pairs(path):
    pairs = []
    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            parts = line.split()
            if len(parts) >= 2:
                pairs.append((parts[0], parts[1]))
    return pairs

def load_corpus_lines(path):
    with open(path, "r", encoding="utf-8") as f:
        return [line.strip() for line in f if line.strip()]

def tokenize_corpus(sentences):
    return [list(jieba.cut(s)) for s in sentences]

def load_or_build_keyed_vectors():
    p = Path(PRETRAINED_W2V_PATH)
    if p.is_file():
        print(f"加载预训练词向量: {p}")
        if p.suffix.lower() == ".kv":
            return KeyedVectors.load(str(p))
        for binary in (False, True):
            try:
                return KeyedVectors.load_word2vec_format(
                    str(p),
                    binary=binary,
                    encoding="utf-8",
                    unicode_errors="ignore",
                )
            except Exception:
                continue
        raise RuntimeError(f"无法解析词向量文件: {p}")
    print(f"未找到预训练文件 {p}, 使用语料训练 Word2Vec: {CORPUS_PATH}")
    sents = load_corpus_lines(CORPUS_PATH)
    if not sents:
        raise FileNotFoundError(f"语料为空: {CORPUS_PATH}")
    tokenized = tokenize_corpus(sents)
    model = Word2Vec(
        sentences=tokenized,
        vector_size=100,
        window=5,
        min_count=1,

```

```

        workers=4,
        sg=1,
        epochs=80,
        seed=42,
    )
    return model.wv
def get_vector(kv, word):
    if word not in kv:
        return None
    return np.asarray(kv[word], dtype=np.float64)
def cosine_sim_vec(a, b):
    if a is None or b is None:
        return None
    a2 = a.reshape(1, -1)
    b2 = b.reshape(1, -1)
    return float(cosine_similarity(a2, b2)[0, 0])
def main():
    kv = load_or_build_keyed_vectors()
    pairs = load_word_pairs(PAIRS_PATH)
    rows = []
    print("\n===== 词对余弦相似度 =====\n")
    for w1, w2 in pairs:
        v1, v2 = get_vector(kv, w1), get_vector(kv, w2)
        sim = cosine_sim_vec(v1, v2)
        if sim is None:
            msg = "(存在未登录词, 无法计算)"
            rows.append((w1, w2, None, msg))
            print(f"{w1} - {w2}\t{msg}")
        else:
            rows.append((w1, w2, sim, ""))
            print(f"{w1} - {w2}\t余弦相似度: {sim:.6f}")
    valid = [(a, b, s) for a, b, s, _ in rows if s is not None]
    if valid:
        print("\n===== 按相似度排序(高 → 低) =====\n")
        for w1, w2, s in sorted(valid, key=lambda x: -x[2]):
            print(f"{s:.6f}\t{w1}\t{w2}")
if __name__ == "__main__":
    main()

```

4. 实验结果（图表与输出）

输出结果为：

F:\Homework\自然语言处理作业\venv\Scripts\python.exe F:\Homework\自然语言处理作业\第四周实验\实验七-词义相似度基础实验.py

Building prefix dict from the default dictionary ...
Loading model from cache C:\Users\23676\AppData\Local\Temp\jieba.cache
未找到预训练文件 pretrained-zh.w2v.txt, 使用语料训练 Word2Vec: ./exp-4.1-corpus.txt
Loading model cost 0.306 seconds.
Prefix dict has been built successfully.

===== 词对余弦相似度 =====

国王 — 皇后	余弦相似度: 0.897633
苹果 — 香蕉	余弦相似度: 0.862986
老师 — 学生	余弦相似度: 0.894615
汽车 — 火车	余弦相似度: 0.732657
高兴 — 快乐	余弦相似度: 0.902362
高兴 — 悲伤	余弦相似度: 0.924558
北京 — 中国	余弦相似度: 0.858878
猫 — 桌子	余弦相似度: 0.766161

===== 按相似度排序（高 → 低） =====

0.924558	高兴	悲伤
0.902362	高兴	快乐
0.897633	国王	皇后
0.894615	老师	学生
0.862986	苹果	香蕉
0.858878	北京	中国
0.766161	猫	桌子
0.732657	汽车	火车

图表结果为:

无图表结果

5. 结果分析

1. 哪些词对的相似度较高？哪些较低？

近义或同类词对如国王皇后、苹果香蕉、老师学生、汽车火车、高兴快乐通常较高；无关或对立词对如猫桌子、高兴悲伤通常较低；北京中国多居中上，因相关但非同义。

2. 结果是否符合语义直觉？请举例说明。

多数符合。例如高兴与快乐应高于高兴与悲伤，苹果与香蕉应高于猫与桌子；若个别不符，多与模型或分词有关。

3. 为什么“高兴-快乐”比“高兴-悲伤”相似度更高？

二者分布与共现更接近，向量更相似；悲伤与高兴语义对立、上下文差异大，故相似度更低。

4. 为什么“北京-中国”相似但不完全相同？

常共现故相关、向量相近；一为城市一为国家级实体，类型与用法不同，故不会等同近义词。

5.是否存在不符合直觉的结果？可能原因是什么？

可能有。原因包括语料偏少或偏科、词频过低、专名切分不当、未登录词，以及词向量反映统计共现而非人工逻辑关系。

6. 实验总结

本实验用词向量表示词语，对任务书中的多组词对计算余弦相似度，输出各对相似度并按高低排序对比。实验流程已按要求做完。